==========
ROBLOX DEVELOPER ARCHIVES: TECHNICAL RESOURCE TR-01
TEXTCHATSERVICE COMPLIANCE CHECKLIST & COMPONENT ARCHITECTURE
Documented by: Hamgram77
Location: Cinnamon Cove Cafe Developer Archives
==========

---
1. OBJECTIVE & APPLICATION

This technical brief outlines the programmatic baseline required to ensure an experience fully satisfies the automated criteria of the platform’s TextChatService engine. Adhering to these strict integration patterns protects independent games from triggering the automated "Failure to properly implement Roblox's TextChatService API" warning, which carries the risk of active publishing fee forfeiture.

---
2. MANDATORY SERVICE CONFIGURATION

To prevent automated QA test bots (such as TCSmodtester) from identifying a custom configuration as a compliance failure, the game's core chat routing must explicitly yield to the modern API architecture.

* Step 1: Property Initialization
  Ensure the legacy Chat service properties are entirely disabled and the modern engine is forced active via the properties panel.
  - TextChatService.ChatVersion must be explicitly set to Enum.ChatVersion.TextChatService.

* Step 2: Safe Server-Side Registration
  Never use custom text replication scripts that bypass internal filtering hooks. All incoming chat strings must resolve through the internal pipeline to ensure filtering is applied universally before rendering.

  [Server Script Service Integration Sample]

```
  local TextChatService = game:GetService("TextChatService")

  -- Explicitly register chat channels via structural initialization
  local function initializeSafeChatChannels()
      local generalChannel = TextChatService:FindFirstChild("RBXGeneral")
      if not generalChannel then
          local newChannel = Instance.new("TextChatChannel")
          newChannel.Name = "RBXGeneral"
```

© 2026 @Hamgram77. All rights reserved. This archive is independent and unaffiliated with Roblox Corporation.

```
        newChannel.Parent = TextChatService
    end
end

initializeSafeChatChannels()
```

---
3. HANDLING CUSTOM CHAT UI SAFELY

If your experience utilizes a custom visual chat log interface (such as a cafe roleplay order log), you must never route text directly from a client's TextBox directly to other players' screens without channeling it through the official text processing callbacks.

* Required Callback Binding:
  Utilize TextChatService.OnIncomingMessage inside local scripts to handle the text rendering stack. This ensures that if the system's filter removes words or flags strings, the engine returns the sanitized text string rather than raw text.

  [Client Local Script Custom UI Integration Sample]

```
local TextChatService = game:GetService("TextChatService")

TextChatService.OnIncomingMessage = function(textResultMessage)
    local properties = Instance.new("TextChatMessageProperties")

    -- Verify message status to guarantee filter compliance before drawing
    if textResultMessage.Status == Enum.TextChatMessageStatus.Success then
        -- Process clean string into your custom UI frame
        return properties
    else
        -- Fallback logic to protect system parameters
        properties.Text = "###"
        return properties
    end
end
```

---
4. PRE-COMPLIANCE VERIFICATION STRATEGY

Before committing a new game version to a paid publishing public slot:

© 2026 @Hamgram77. All rights reserved. This archive is independent and unaffiliated with Roblox Corporation.

1. Run a local multi-client simulation in Roblox Studio.
2. Verify that TextChatService completely replaces the old Chat folder
   in the Explorer window.
3. Ensure no local console script prints out warning logs referencing
   unhandled text packets during player messaging routines.
==========

© 2026 @Hamgram77. All rights reserved. This archive is independent and unaffiliated with Roblox Corporation.